

EXPERIMENTAL DATA MANAGEMENT SYSTEM

Milestones 5 & 6 Documentation

1. MILESTONE 5: CONCURRENT & SCALABLE SYSTEM

Objective

The goal of Milestone 5 was to transform the system from a standard data processing application into a high-performance concurrent system capable of handling large datasets efficiently.

What Changed

1. Multiprocessing for Statistical Computation

Mean and variance calculations were parallelized using CPU cores.

2. Threading (Task Queue System)

A ProcessingQueue was introduced to simulate concurrent task execution.

3. Asynchronous File Operations

CSV and JSON exports were converted to async functions.

4. Performance Benchmarking

Sequential vs parallel execution time comparison added.

What Failed

1: Race Conditions in Threading

Tasks in the queue were occasionally accessed simultaneously.

2: Complexity in Async + Thread Mixing

Combining asyncio with ThreadPoolExecutor caused confusion in execution flow.

3: ProcessingQueue Not Defined (Integration Error)

During system integration, the class was missing from the final structure.

Why it Failed

Python threads share memory → unsafe access without locks

Async and threading operate on different execution models

Code refactoring led to accidental omission of components

How it was Fixed

1: Thread Safety

Added threading.Lock() to protect shared queue data.

2: Controlled Concurrency Design

Separated concerns:

threading → task processing

asyncio → file operations

3: System Re-integration

Restored ProcessingQueue class and structured code in correct order

Outcome of Milestone 5

Improved performance for large datasets

Introduced real concurrency concepts

System became scalable and modular

2. MILESTONE 6: RESEARCH & INTELLIGENT SYSTEM

Objective

To extend the system into a research-level intelligent analytics platform by introducing statistical reasoning and anomaly detection.

What Changed

1. Anomaly Detection Module

Z-score based statistical outlier detection

2. Statistical Intelligence Layer

Mean and standard deviation computed per variable

3. Classification System

Normal / Borderline / Outlier labeling

What Failed

1: Large Output Volume

The anomaly system printed results for every measurement, making output difficult to interpret.

2: Division by Zero Risk

Variables with constant values caused $\text{std} = 0$.

3: High Computation Overhead

Full dataset traversal for anomaly detection increased processing time.

Why it Failed

The system processes all raw measurements without sampling

Statistical variance can be zero in uniform datasets

No aggregation layer was initially implemented for reporting

How it was Fixed

1: Structured Output Design

Results grouped per variable for better readability

2: Safe Statistical Handling

Added condition:

if $\text{std} = 0 \rightarrow \text{z-score set to 0}$ (safe fallback)

3: Efficient Generator Usage

Used measurement generator to avoid memory overload

Outcome of Milestone 6

Introduced intelligent data interpretation

Enabled automatic anomaly detection

Upgraded system from “data processor” to “decision-support system”